

Git 기본 명령어 10가지

1~2. 초기 사용자 설정

내 컴퓨터에서 Git을 처음 쓸 때, "이 코드는 내가 짰다"라고 명함(이름과 이메일)을 등록하는 명령어입니다.

- ☆ **git config --global user.name "내이름"** : 내 이름을 전역(Global) 설정으로 등록합니다.
- ☆ **git config --global user.email "내이메일@example.com"** : 깃허브 계정에 등록된 이메일을 입력합니다.

3. 프로젝트 시작

- ☆ **git init** : 현재 내가 작업 중인 폴더를 Git이 관리하는 프로젝트(저장소)로 초기화합니다. 이 명령을 치면 눈에 보이지 않는 .git 폴더가 생기며 버전 관리가 시작됩니다.

4. 원격 저장소(GitHub) 연결

- ☆ **git remote add 원격이름 주소** : 내 컴퓨터의 로컬 저장소와 인터넷 세상의 깃허브 저장소를 연결합니다. 보통 원격 이름은 origin을 관례로 사용합니다.
사용법: **git remote add origin https://github.com/유저명/InstaFrame.git**

5~6. 상태 확인 및 무대 올리기

- ☆ **git status** : 현재 어떤 파일이 수정되었고, 어떤 파일이 Git에 기록될 준비가 되었는지 상태를 보여주는 '스캐너'입니다. 수시로 입력해 보면 좋습니다.
- ☆ **git add 파일명** : 수정된 파일을 촬영 대기 무대(Staging Area)에 올립니다. 폴더 안의 모든 수정 사항을 한 번에 올리려면 주로 점(.)을 찍어 사용합니다.
사용법: **git add .** (모든 변경 파일 무대에 올리기)

7. 버전에 도장 찍기

- ☆ **git commit -m "메시지"** : 무대에 올라간 파일들을 하나의 완벽한 '버전'으로 기록하고, 저장소에 박제합니다.
사용법: **git commit -m "Fix: Supabase Storage 업로드 로직 구현"**

8. 깃허브로 밀어 올리기

- ☆ **git push 원격이름 브랜치명** : 내 컴퓨터에만 기록된 커밋(버전)들을 연결된 인터넷 깃허브 저장소로 업로드합니다.
사용법: **git push origin main (또는 master)**

9. 통째로 복사해 오기

☆ **git clone 주소** : 깃허브에 있는 다른 사람의 프로젝트나 내 프로젝트를 내 컴퓨터로 통째로 다운로드하면서 동시에 Git 저장소로 연결까지 완료합니다.

사용법: **git clone https://github.com/유저명/프로젝트.git**

10. 최신 코드 당겨오기

☆ **git pull 원격이름 브랜치명** : 깃허브 저장소에 올라와 있는 최신 변경 사항을 내 컴퓨터로 가져와 현재 내 코드와 합칩니다.

사용법: **git pull origin main**

실전에서 무조건 쓰게 되는 추가 필수 명령어 4가지

11. git log - 버전 역사책 보기

내가 지금까지 어떤 커밋들을 남겼는지 히스토리를 내림차순으로 보여줍니다. 커밋의 고유 번호(Hash)와 작성자, 날짜, 커밋 메시지를 확인할 수 있습니다.

- 사용법: **git log** (터미널에서 빠져나올 때는 키보드 q를 누릅니다.)

12. git branch - 평행우주(가지) 만들기

현재 작동 중인 안정적인 메인 코드(main)를 건드리지 않고, 새로운 기능(예: "Supabase 전환 테스트")을 안전하게 독립적으로 개발하기 위해 새로운 가지를 뽀는 명령어입니다.

- 사용법: **git branch 브랜치이름** (브랜치 생성)

- 확인: **git branch** (현재 존재하는 브랜치 목록 보기, 초록색이 현재 내 위치)

13. git checkout (또는 git switch) - 평행우주 이동하기

내가 만든 다른 브랜치로 이동하거나, 과거의 특정 커밋 시점으로 코드를 되돌려가볼 때 사용합니다. 최근 Git 버전에서는 브랜치 이동 전용으로 switch라는 명령어가 추가되었습니다.

- 사용법: **git switch 브랜치이름** (해당 브랜치로 내 코드 환경이 바뀜)

- 예시: **git switch feature/supabase**

14. git merge - 우주 합치기

새로운 브랜치(예: feature/supabase)에서 기능 구현과 테스트가 완벽히 끝나면, 이를 다시 서비스의 중심인 main 브랜치로 가져와서 합쳐야 합니다.

- 사용법: 우선 중심 브랜치로 이동합니다: **git switch main**

- 가져올 브랜치를 합칩니다: **git merge feature/supabase**